

Student Name:

Student ID:

Habib University
Dhanani School of Science and Engineering
Computer Architecture
EE 371 / CS 330 / CE 321 (Fall 2025)

Midterm 1

Date: 18th Oct, 2025

Total marks: 100

Obtained Marks:

Total time: 80 mins

Instructions:

1. All questions should be answered **with ink pen**.
2. Smart watches, laptops, and similar electronics are **strictly NOT allowed**.
3. Answer sheets should contain all steps, working, explanations, and assumptions.
4. Write your name and HU ID on all sheets.
5. Turn in your question paper along with your answer sheets.
6. You are not allowed to ask/share your method or answer with your peers. The work submitted by you is solely your own work. Any violation of this will be the violation of HU Honor code and proper action will be taken as per university policy if found to be involved in such an activity.

CLO Assessment:

This exam assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	✓
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	✓
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	
CLO 4	<i>Analyze the</i> performance of cache operations	

Student Name:

Student ID:

Q1: All parts are assessing CLO 1 [Total 50 points]

Assume that the base address of a word array having 100 elements is stored in register x1. We want to assign -100 to each element of the word array using RISC-V assembly. Remember that RISC-V only supports **bge**, **bgeu**, **beq**, **bne**, **blt** and **bltu** instructions for branching.

- a) Write the program using array based implementation. (10 points)**

Sol:

```
addi x10, x0, 0           // loop variable
addi x11, x0, 100         // max index
addi x12, x0, -100        // set value
loop:slli x13, x10, 2      // word array, 4 bytes offset
add x14, x13, x1          // address of array[i]
sd x12, 0(x14)            // array[i] = -100
addi x10, x10, 1          // index = index + 1
blt x10, x11, loop
```

- b) Re-write the program using pointer based implementation. Assume now x1 is pointer to word array (10 points)**

Sol:

```
addi x10, x0, 0           // loop variable
addi x11, x0, 100         // number of elements
addi x12, x0, -100        // set value
slli x13, x11, 2          // address of last element
loop:sd x12, 0(x1)         // *ptr = -100
addi x1, x1, 4            // ptr = ptr + 4
blt x1, x13, loop
```

Student Name:

Student ID:

- c) Considering various factors of the CPU performance equation, in your opinion, which program will have better performance in terms of CPI. Restrict your answer to 5 lines **(5 points)**

Sol: Pointer based version since we have lesser instructions (instruction count) inside loop.

- d) Consider this RISC-V assembly code

```
addi x5, x0, -100
```

```
addi x6, x0, 1
```

```
loop:
```

```
add x5, x5, x6
```

```
bne x5, x0, loop
```

Calculate CPI for this code. Assume `addi` takes 2cc, `add` takes 1cc and `bne` takes 3cc. **(9 points)**

Sol:

Instruction count and clock cycles:

```
addi → 2      cc → 2*2 = 4
```

```
add → 100     cc → 1*100 = 100
```

```
bne → 100     cc → 3*100 = 300
```

IC_{total} = 202, cc_{total} = 404

CPI = cc_{total}/IC_{total} = 404/202 = 2.0

Student Name:

Student ID:

- e) Write how each of these effect CPU performance (3 lines max, extra lines will NOT be graded)

i. Algorithm (2 points)

IC and CPI.

The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more divides, it will tend to have a higher CPI.

ii. Compiler (2 points)

IC and CPI.

The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions.

iii. Instruction Set Architecture (hardware) (2 points)

IC, CPI and cc.

The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.

- f) Consider the following three functions, calling one another.

```
main(){
    i = i + 5;
    funct_1();
    return;
}

funct_1(){
    j = j + 6;
    funct_2();
    return;
}

funct_2(){
    k = k + 7;
    return;
}
```

Assume we want to convert the following C functions into RISC-V assembly, adhering strictly to the **RISC-V procedure calling convention**. Also, assume that the variables `i`, `j`, and `k` can be stored in **temporary registers**, and no callee-saved registers are needed for them. With these assumptions, does any of the function requires stack? If no, justify your answer. If yes, write assembly code for the function which needs stack. **(10 points)**

Sol:

Strictly using RISC-V procedure convention, we need stack for `funct_1()`.

```
funct_1:  addi sp, sp, -8
         sd   x1, 0(sp) # since funct_1 is calling funct_2
         addi x6, x6, -6
         jal  x1, funct_2      # x1 over-written here
         ld   x1, 0(sp)
```

Student Name:

Student ID:

```
addi sp, sp, 8
jalr x0, 0(x1)
```

Q2: All parts are assessing CLO 2 [Total 50 points]

IEEE 754 single precision floating point format

1-bit sign	8-bit exponent	23-bits fractional part
------------	----------------	-------------------------

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} \rightarrow \text{Normal numbers}$$

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}} \rightarrow \text{Subnormal numbers,}$$

- a) Write the largest sub-normal number in 32-bit binary format. Remember that for sub-normal numbers, exponent is zero. **(5 points)**

0 00000000 111111111111111111111111

- b) Convert 0x40400000 single precision floating point number to decimal. **(5 points)**

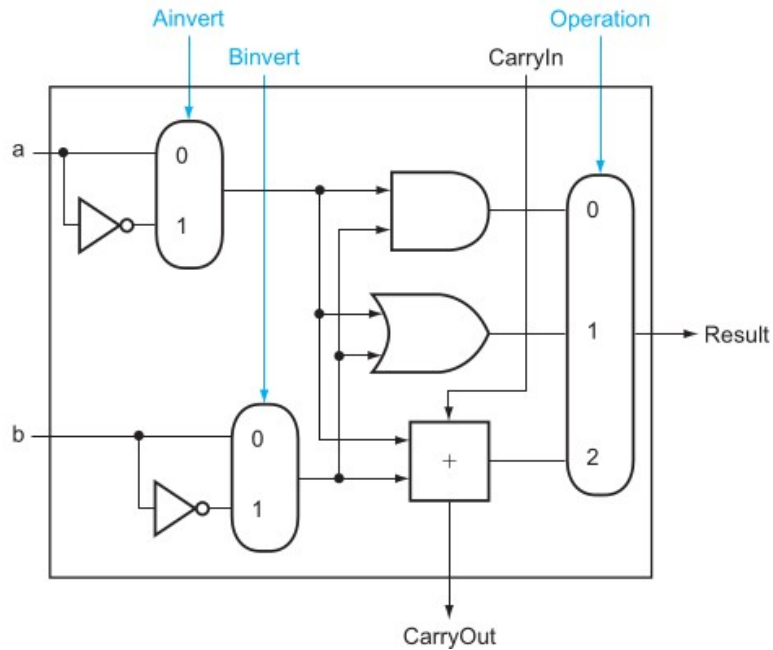
Sol: 3.0

Student Name:

Student ID:

- c) Design and draw a logic circuit for a 1-bit ALU that supports the following operations: AND, OR, NAND, NOR, ADD, and SUB. Use basic logic gates, multiplexers (MUXes), and a 1-bit full adder in your design. Clearly label all signals with their names and bit widths. Indicate how operation selection is handled (e.g., using control signals). **(10 points)**

Sol:



- d) Consider the given RISC assembly code:

```
add x5, x0, value1      # value1 = 100, PC = 100 at cc = 0
```

```
add x6, x0, value2      # value2 = 200
```

```
label_equal:
```

```
ld x7, 0(x5)            # value at value1 (10)
```

```
ld x8, 0(x6)            # value at value2 (20)
```

```
add x9, x7, x8
```

```
sub x10, x8, x7
```

```
and x11, x7, x8
```

```
sd x9, 0(x12)           # x12 = 300
```

```
beq x7, x7, label_equal
```

Write correct values of control and signals and current PC for the given code. Assume we have a single cycle data-path/processor. For rs1, rs2 and rd, write values in binary whereas ALU oprnd 1, 2 and result should be written in decimal **(30 points)**

ALUop → 10 for R type, 00 for ld/sd and 01 for branch

Student Name:

Student ID:

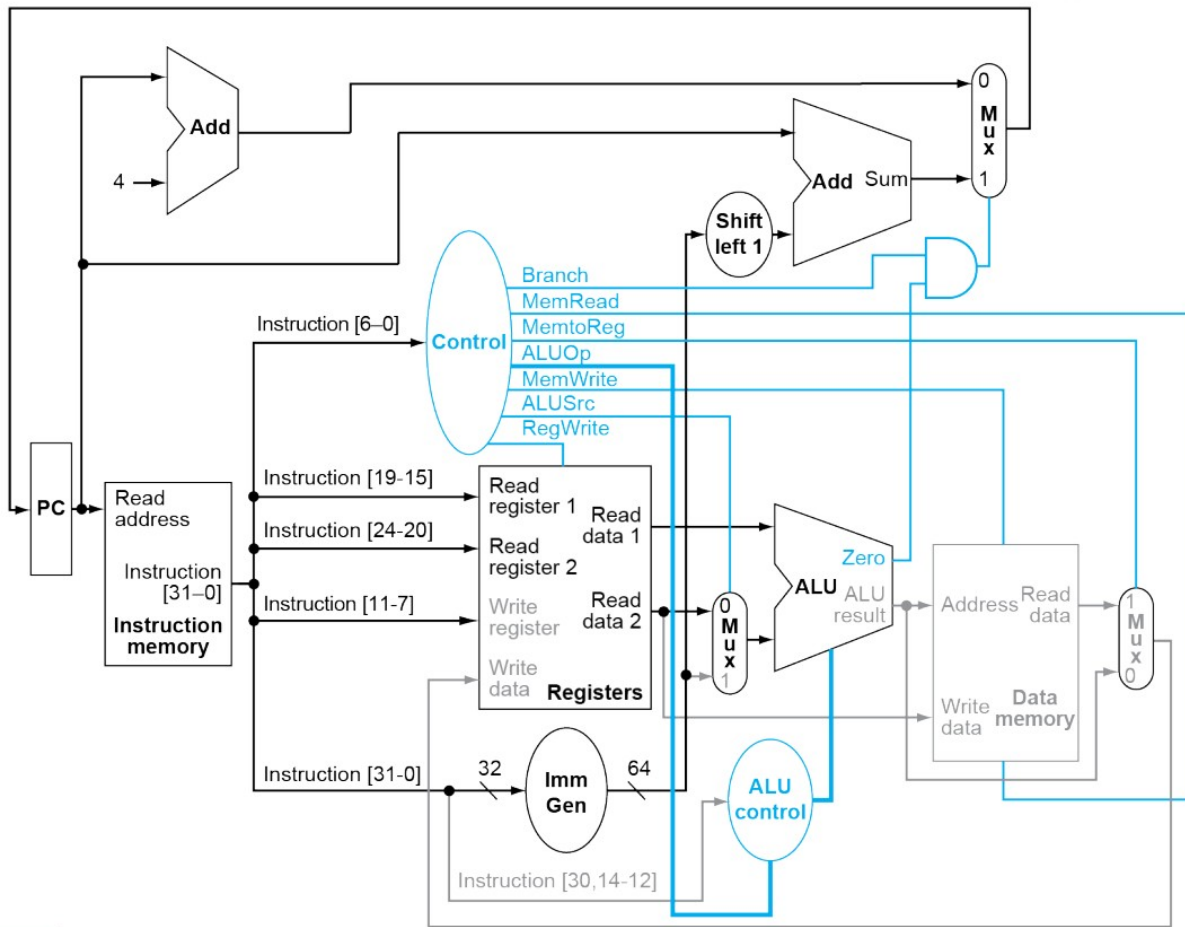
Sol:

	cc 0	cc 1	cc 2	cc 3	cc 4
Instruction	addi x5	addi x6	ld x7	ld x8	add x9
PC	100	104	108	112	116
RegWrite	1	1	1	1	1
ALUSrc	1	1	1	1	0
ALUOp	00	00	00	00	10
MemWrite	0	0	0	0	0
MemRead	0	0	1	1	0
Branch	0	0	0	0	0
MemtoReg	0	0	1	1	0
ALU oprnd 1	100	200	100	200	10
ALU oprnd 2	0	0	0	0	20
ALU result	100	100	100	200	30

	cc 5	cc 6	cc 7	cc 8	cc 9
Instruction	sub x10	and x11	sd x9	beq x7	ld x7
PC	120	124	128	132	108
RegWrite	1	1	0	0	1
ALUSrc	0	0	1	0	1
ALUOp	10	10	00	01	00
MemWrite	0	0	1	0	0
MemRead	0	0	0	0	1
Branch	0	0	0	1	0
MemtoReg	0	0	x/0	x/0	1
ALU oprnd 1	20	10	300	10	100
ALU oprnd 2	10	20	0	10	0
ALU result	10	0	300	True	100

Student Name:

Student ID:



Preserved	Not preserved
Saved registers: x8-x9, x18-x27	Temporary registers: x5-x7, x28-x31
Stack pointer register: x2(sp)	Argument/result registers: x10-x17
Frame pointer: x8(fp)	
Return address: x1(ra)	
Stack above the stack pointer	Stack below the stack pointer

$$\text{CPU clock cycles} = \sum_{i=1}^n (\text{CPI}_i \times C_i)$$

$$\text{CPI} = \frac{\text{CPU clock cycles}}{\text{Instruction count}}$$

```

jal x1, ProcedureAddress    // jump to
                             ProcedureAddress and write return address to x1

jalr x0, 0(x1)             // branch back to calling routine

```